```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

columns = [
“buying”,
"maint",
"doors",
"persons",
"lug boot",
"safety",
"class"

]

data = pd.read_csv("car_evaluation.csv", header=None, names=columns)

print (data.head())
print("Dataset shape:", data.shape)
print("Columns:", data.columns.tolist())

print (data.isnull().sum())

label_encoders = {}
for column in data.columns:
encoder = LabelEncoder()
data[column] = encoder. fit_transform(data[ column] )

label_encoders[column] = encoder

print (data.head() )

X = data.drop("class", axis=1)
y = data["class"]
```

```
print("Features:", X.shape)
print("Target:", y.shape)
print("Car safety classes:", np.unique(y))

X_train, X_test, y_ train, y_test = train_test_split(
X, Ys
test_size=0.2,
random_state=42,
stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

rf_model = RandomForestClassifier(
n_estimators=100,
random_state=42

)

rf_model.fit(X_train, y_train)

print("Random Forest model trained successfully!")

y_pred = rf_model.predict(X_test)
print("Actual values: ", y_test.iloc[:10].values)
print("Predicted values:", y_pred[:10])

]

accuracy = accuracy_score(y_test, y_pred)

print("Random Forest Accuracy: {:.2f}%".format(accuracy * 100@))

cm = confusion_matrix(y_test, y_pred)


plt.figure(figsize=(6, 4))
plt.imshow(cm, cmap="Blues")

plt.title("Random Forest Confusion Matrix")

plt.xlabel("Predicted Class")
```

```
plt.ylabel("Actual Class")
plt.colorbar()

plt.show()

print(classification_report(y_test, y_pred) )

new_car = pd.DataFrame([{
"puying": "vhigh",
"maint": "high",
"Doors":"4”
"persons": "4",
"lug boot”: “small”,
"safety": "low" ])

new_car[column] = label_encoders[column].transform(new_car[column] )
prediction = rf_model.predict(new_car[X.columns ])

class_encoder = label_encoders["class" ]
print("Predicted car safety:", class _encoder.inverse_transform(prediction) [@])

Predicted car safety: unacc

print ("RESULT:")
print("The Random Forest Classifier was implemented" )
print("to predict car safety using the Car Evaluation dataset.")
```